UNIT-3 - SOFTWARE DESIGN

Software design:-

Software design is model of software which translates the Requirements into finished software product in which the details about software data structures, architecture, Interface and components that are necessary to implement the system are given.

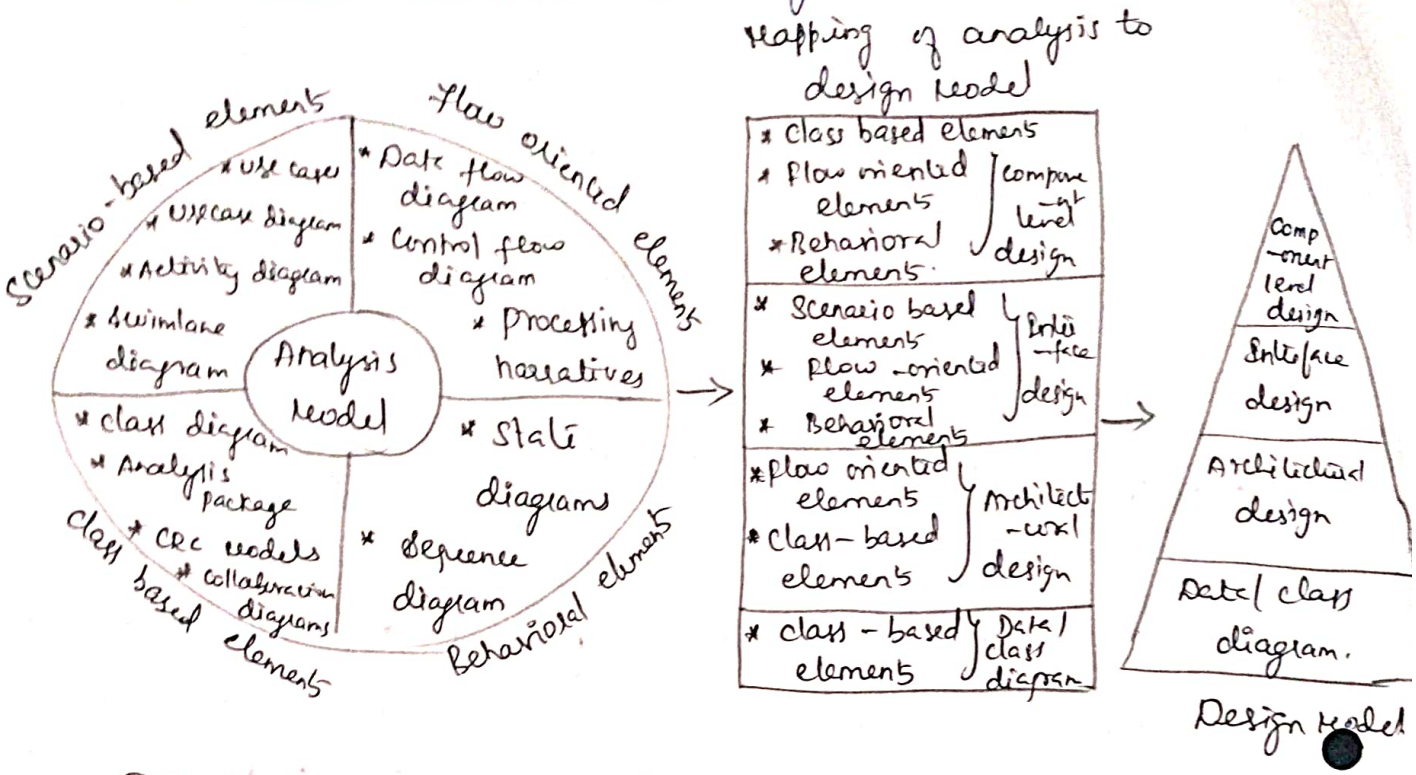
Designing within the context of Software Engineering:-

- Software design is a vital activity during the process of Software development.
- The first task is to analyse and Identify the Requirement. Based on the analysis the Software design is developed.
- This design serves as a basic for code generation and testing.
- Intermediate process which translates the analysis model into design model.

The four types of elements of analysis model are scenario based elements, class based elements, behavioral elements and flow oriented elements.

- Scenario based elements Analysis Various models such as use case diagram, activity diagram or swimlan diagrams are created.
- class base analysis the class diagram are creat
- During flow oriented analysis the Data flow diagrams are created.
- Behavioral analysis can be done using Stat and Sequence diagram.

The flow oriented elements and class based elements are used to create architectural design.



Translating Analysis model into the design Model.

- The Scenario based elements, flow oriented elements and behavioral elements are used to create Interface design.
- The class based elements, flow oriented elements and behavioral elements are used to create component level design.

Design process:-

- Software design is an iterative process using which the user requirements can be translated into the software product.
- Initial stage of software is represented as Abstract view.

Characteristic of Good Design

1. The good design should implement all the Requirements specified by the customer.
2. The design should be simple enough so that the

Software can be understood easily by the developer, tester and customer.

3. The design should be comprehensive. That means it should provide complete picture of the software.

Quality guideline:-

- The goal of any software design is to produce high quality software.
- Evaluate quality of software there should be predefined rules or criteria that need to be used to assess the software product.

The quality guideline are as follows:-

* The design should be created using architectural style and pattern.
* Each components of design should posses good design characteristic
* The implementation of design should be evolutionary. so that testing can be performed at each phase of implementation.
* In the design data, architecture, interface and components should be clearly represented.
* The design should be modular.
- The data structure should be appropriately choosen for the design of specific problems.
* The components should be used in the design so that functional independency can be achieved in the design.

6) Using the information obtained in software requirement analysis the design should be created.

7) The interface in the design should be such that the complexity between the connected components of the system gets reduced

8) Every design of the software system should convey its meaning appropriately and effectively.

Quality Attributes (FURPS model) ⇒ Functionality, Usability, Reliability, performance and supportability)

The design quality attributes popularly known as FURPS is a set of criteria developed by Hewlett and packard.

Quality Attributes	Meaning.
① Functionality	- Assessing the set of features and capabilities of the functions. - Function should be general and should not work only for particular set of inputs. - Security aspect should be considered while designing the function.
② Usability -	The usability can be assessed by knowing the usefulness of the system.
③ Reliability -	Reliability is a measure of frequency and severity of failure. Repeatability refers to the consistency and repeatability of the measure.

Reliability - The Mean Time to Failure (MTTF) is a metric that is widely used to measure the product's performance and reliability.

performance - It is measure that represents the response of the system.

- Measuring the processing speed, memory usage, respons time and efficiency.

Supportability - It is also called maintainability. It is ability to adapt the enhancement or changes made in the software.

Design concepts:-

The software design concept provides a framework for implementing the right software.

Following are certain issues that are considered while designing the software -

* Abstraction
* Modularity
* Architecture
* Refinement
* pattern
* Information Hiding
* Functional Independence
* Refactoring
* Design Classes.

Abstraction:-

- The Abstraction means an ability to cope up with the complexity.

- At the higher level of abstraction, the solution should be stated in broad terms and in the lower level more detailed description of the solution is given.

- procedural Abstraction and data abstraction are created

- The procedural abstraction give the named sequence of instructions in the specific function.

- Implementation details are hidden.

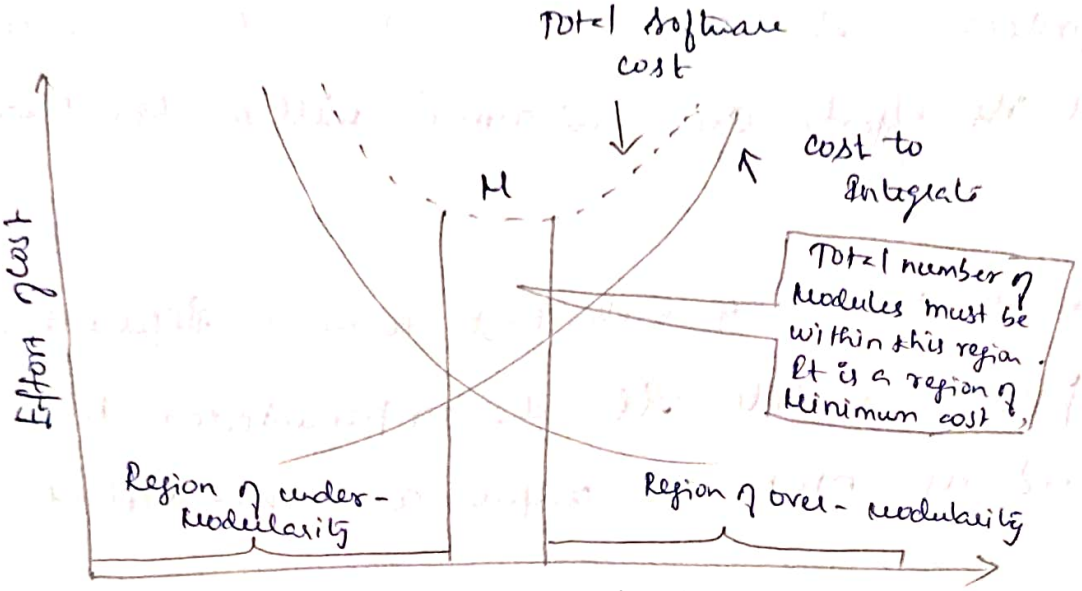
For Ex - Searching the records, (i.e enter the name, compare each name of the record against the entered one, 'If a match is found then declare 'success!! otherwise declare 'name not found').

- In data abstraction, the collection of data objects is represented. For ex - The record consists of various attributes such as record ID, name, address and designation.

Modularity:-

* The software is divided into separately named and addressable components that called as modules.

* Monolithic software is hard to grasp for the software engineer, hence it has now become a trend to divide the software



-overmodularity or the undermodularity while developing the software product must be avoided. It is denoted by 'M'

Meyer: define five criteria that enable us to evaluate a design method with respect to its ability to define an effective modular system.

Modular decomposability - A design method provides a systematic mechanism for decomposing the problem into sub-problems. This reduce the complexity of the problem and achieved modularity

Modular composability - A design method enables existing design components to be assembled into a new system.

Modular understandability - A module can be understood as a standalone unit. It is easier to build and change.

Modular Continuity - Small changes to the system requirements result in changes in individual modules, rather than system-wide change.

Modular protection: Th aberrant condition occurs within module and its effects are constrained within the modules.

Importance:-

Due to modularity each task forms a separate, distinct program module. At the implementation time each module and its inputs and outputs are well-defined.

Architecture:-

- Architecture means representation of overall structures of an integrated system.
  - Data of the structure are used by various components
  - This components are called system elements.
  - Architecture provides the basic framework.

In architectural design various system models can be used and these are,

Model	Functioning.
Structural model	Overall architecture of the system can be Represented.
Framework model	Shows the architectural framework and corresponding Applicability
Dynamic model	Shows the reflection of changes on the System due to external events.
process model	Sequence of processes and their functionality is Represented.
Functional model.	The functional Hierarchy occuring in the system.

Refinement:-

- Refinement is actually a process of elaboration.
- Stepwise Refinement is a top-down design strategy.
- The process of program Refinement is analogous to the process of Refinement and partitioning that is used during Requirement Analysis.
- Abstraction and Refinement are complementary concepts.
- The major difference is that in the abstraction low-level details are suppressed. Refinement helps the designer to elaborate low-level details.

Pattern:

Design pattern act as a design solution for a particular problem occurring in a specific domain.

* pattern can be reusable
* pattern can be used for current work
* pattern can be used to solve similar kind of problem with different functionality

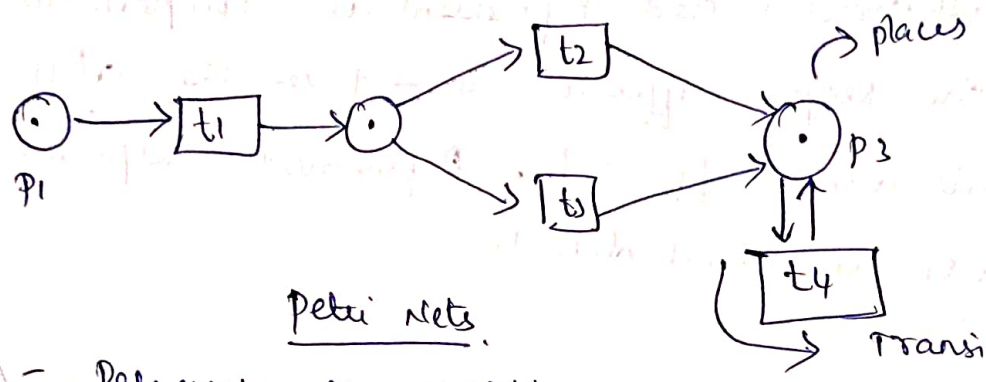
Information Hiding:-

- The information contain in one module it cannot be accessible to the other module.
- Only limited information can be hiding and passed to other module.
- Adv is testing and maintenance.

Petri Nets:-

- The petri nets are invented by Carl Adam petri in 1962.
- The petri nets can be rigorously used to define the system. The petri nets are more popular used to for distributed systems and systems with Resource sharing.

Three types of components - places (circles), transitions (rectangles) and arcs (arrows)



Petri Nets.

Places - Represent the possible state of the system.

Transitions - Cause the change in the states

Arc - Connects a place with transition.

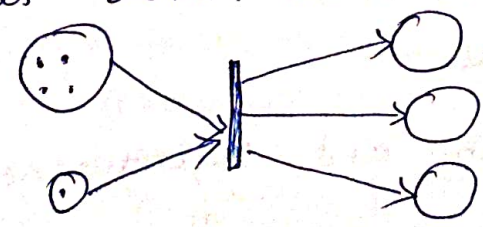
For Ex:-

Equivalent Notation is used is a solid bar. It is used to denote transitions.

- Tokens Represents the dynamic objects.
- States having black dots are called marked petri token.
- Single dot inside the circle that it represent the one unit of certain Resources.

Firing a transition:-

Transitions takes input from the input places and produces tokens in the output places.



Firing Transitions.

- One can make changes in the desired module without affecting the other module.

Functional Independence:

- The functional Independence can be achieved by developing the functional modules with single-minded approach.

- Independent modules are easier to maintain with reduced error propagation.

- It is a key to good design and design is the key to software quality.

- Achieving the effective modularity

- Two quality criteria - cohesion and coupling.

cohesion:

- with the help of cohesion the information hiding can be done.

- A cohesive perform only "one task" in software procedure with little Interaction with other modules.

Different types of cohesion are:

1. Coincidentally cohesive - Set of task are related with each other loosely then such modules are called coincidentally cohesive.

2. Logical cohesive - That perform the task logically related with each other.

3. Temporal cohesion - The task need to be executed in some specific time span

4. procedural cohesion - when processing elements of a module are related with one another & must be executed in some specific order.

5. Communicational cohesion - When the processing elements of a module share the data then such module is communication cohesion.

Coupling:-

- Coupling effectively represent how the modules can be "connected" with other module
- Measure of interconnection among modules in a program structure.
- Depends on the interface complexity between modules.
- It should reduce or avoid change impact and ripple effects. It should also reduce the cost in program changes, testing and maintenance.

Various types of coupling are:

1. Data coupling - parameter passing or data interaction.
2. Control coupling - share related control data in module control coupling
3. Common coupling - common data or global data is shared among the modules.

4) Content coupling - When one module make use of data or control information maintained in another module.

Refactoring:-

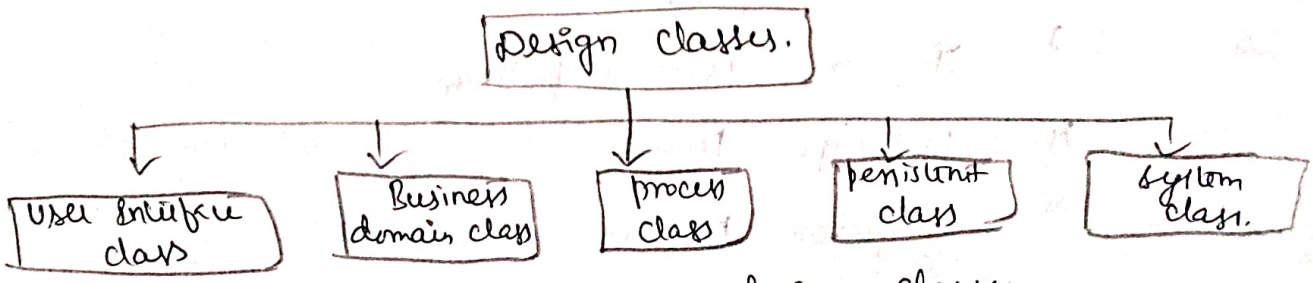
- Refactoring is necessary for simplifying the design without changing the function or behaviour.
has defined as
Fowler :- The process of changing a software system in such a way the external behaviour of the design do not get changed, however the internal structure get improved.

Benefits of Refactoring are,

1. The redundancy can be achieved
2. Inefficient Algorithms can be eliminated
3. Inaccurate data structures can be removed or replaced.
4) Other design failures can be rectified.

Design classes:-

Design classes are defined as the classes that describe some elements of problem domain, focus on various aspects of problem from user's point of view.



Types of design classes.

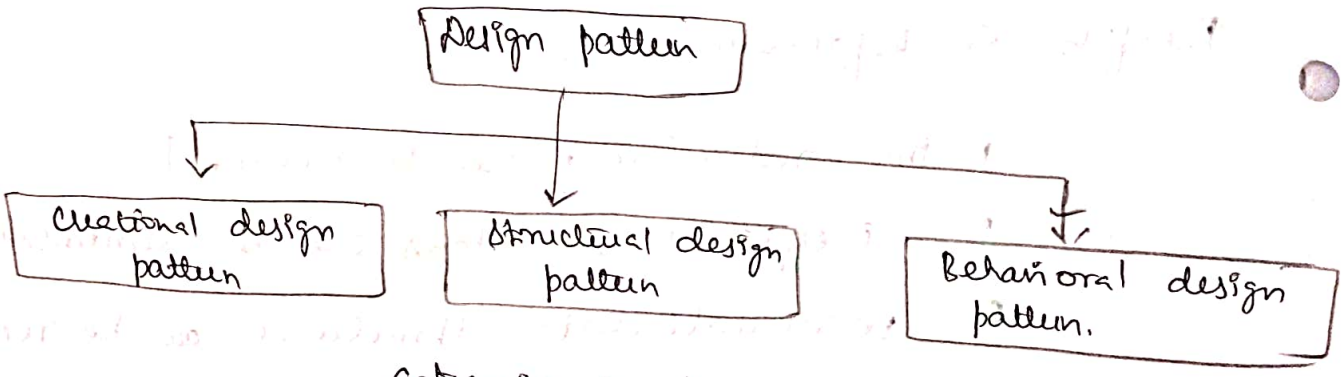
Design patterns:

Design pattern is general reusable solution to commonly occuring problem within a given context in software design.

pattern has four essential elements.

1. pattern Name
2. problem
3. solution
4. consequence.

Design pattern is divided into three categories - Creational pattern, structural pattern and behavioral pattern.



Categories of design pattern.

Creational pattern:-

- Creation design pattern are the design pattern that are used for object creation.
- Five well design pattern are,
  1. abstract factory pattern
  2. Builder pattern
  3. Factory method pattern.
  4. prototype pattern
  5. Singleton pattern.

Structural pattern:

In software engineering, structural design patterns are design patterns that ease the design by identifying a simple way to realize relationship between entities.

There are various design patterns that are the part of structural design pattern.

1. Bridge
2. Adapter
3. Composite
4. Decorator
5. Facade
6. Flyweight
7. private class data
8. proxy.

Behavioral pattern:-

- To identify the common communication patterns between objects and realize these patterns.

Various design patterns that are the part of behavioral design pattern.

1. Chain of Responsibility
2. Command
3. Interpreter
4. Iterator
5. Mediator
6. Null object
7. Memento
8. Observe
9. State
10. Strategy
11. Template method
12. Visitor.

Adapter (*) AU

Sometime we come across the situation where two components are working well functionally but they can not work together because of some reason.

In object oriented design the adapter pattern is adapting between classes and objects.

Problem - How to resolve incompatible interfaces, How to provides the a stable interface to the components having different interfaces?

Soln

Convert the original interface of components into another interface through intermediate adapter object.

This pattern is also known as wrapper. This is a GoF pattern.

There are two types of adapter -

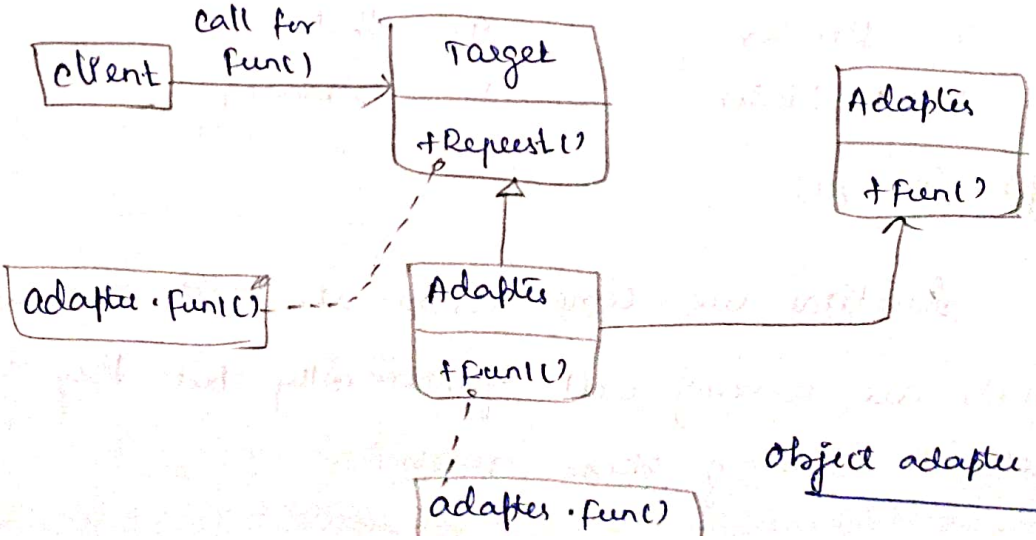
1. Object adapter 2. Class Adapter.

Object Adapter:

- In this type of adapter pattern the adapter contains the instance of class it wraps.

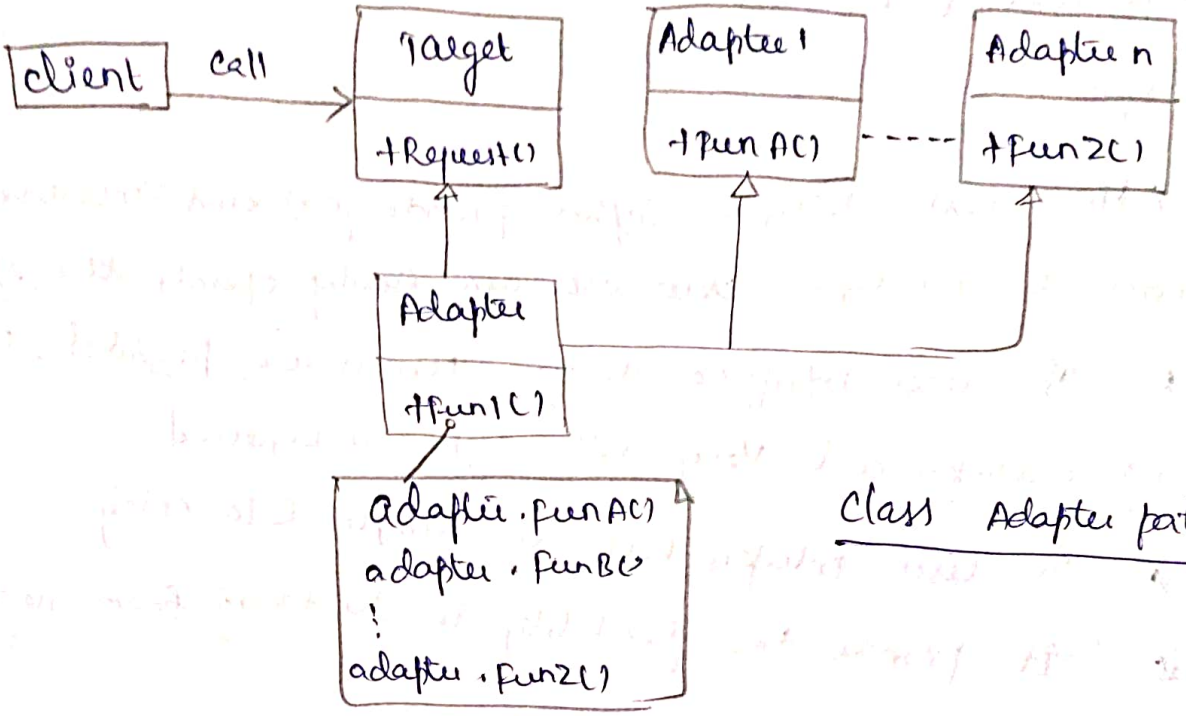
- When the client wants some task to get fulfilled it make a request to the Target. The Adapter inherits the target and at the same time holds the instance of Adaptee. The Adapter who hides the Adaptee's interface from the client.

Class Adapter:-



Object adapter pattern.

The Adapter uses multiple polymorphic Interface of Adaptee Hence the number of request of Adaptee1, Adaptee2 ... AdapteeN can be Invoked by the Adapter.



Class Adapter pattern.

Example

Adapter pattern are windows Adapter in Swing. API or JDBC, ODBC API

Benefits

1. Two Compatible objects or classes can be Interfaced using adapter pattern.
2. Reduces coupling between the objects.

User Interface Design:-

- Any computer based system require two things : Is computational ability and functionality.
- The main purpose is to have effective communication medium between the computerized system and the user.
- This kind of Interface design is necessary because of many reason such as : Software is difficult to use,

the use of software forces the user to make mistakes due to lack of understanding about the system.

- While designing for user Interface the very first step is to identify user, tasks and environmental techniques.

Advantages:-

* The user Interface system provide fast and intuitive Interaction to the user. New user can easily operate the system.
* The user Interface design, menus are provided, it avoid typing mistakes and very little typing is required.
* The user Interface helps in simple data entry.
* It provide the flexibility in switching from one task to another.

Golden Rules:-

Theo Mandel has proposed three golden rules for user Interface design,

1. place the user in control
2. Reduce the user's memory load
3. Make the Interface Consistant.

Place the user in control:-

While Analysing any Requirement during Requirement Analysis the user often demands for the system which will satisfy user Requirements and help him to get the things done.

1. Define the Interaction modes in such a way that user will be restricted from doing the unnecessary actions.

* The Interaction should be flexible
* provide the facility of 'undo' or 'Interruption' in user Interaction.
* Allow user to customize the Interaction
* Hide technical details from the user.
* The objects appearing on the screen should be Interactive with the user.

Reduce the User's Memory Load:

- If the user Interface is good then user has to remember very less.
- The system remembers more for the user and ultimately it assists the user to handle the computer based system.

Following principles suggested by Mandel to reduce the memory load of the user.

1. Do not force the user to have short term memory
2. Establish meaningful defaults
3. Use Intuitive shortcuts
4. The visual layout of the Interface should be realistic
5. Disclose the information gradually.

Make the Interface Consistent:-

The user Interface should be consistent. This consistency can be maintained at three level such as,

* The Visual Information (all screen layouts) should be consistent throughout and it should be as per the design standards.

* There should be limited set of input holding the non-Conflicting Information.
* The Information flow transiting from one task to another should be consistent.

Principles for consistant Interface Design,

1. Allow User to direct the current task into Meaningful manner.
2. Maintain consistency across family of product
3. If certain standard are maintained in previous model of applications do not change it until and unless it is necessary.

User Interface Analysis and Design:-

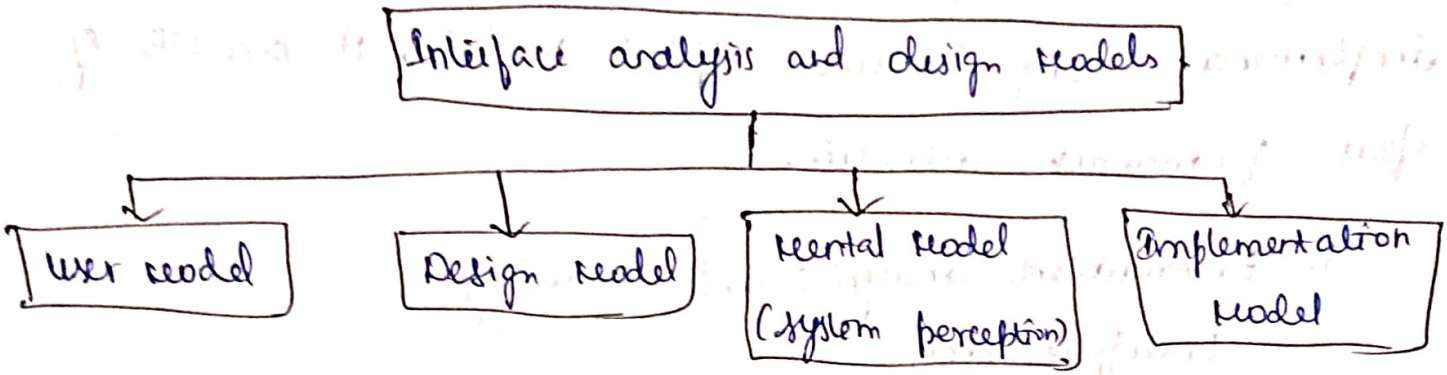
User Interface analysis and design can be done with the help of following steps:

- Create different models for system functions
- Prepare all interface designs by solving various design issues
- Implement design model
- Evaluate the design from end user to bring Quality in it.

two classes,

- Interface analysis and design models
- The process

Interface Analysis and Design Model:-



Analysis and design model.

User model - To design any user interface it is a must to understand the user who is using the system.

The profile of the user is prepared by considering age, sex, educational, economical and cultural background.

Design model:

It consists of data, architectural, interface and procedural representation of the software.

Mental model (System perception)-

The user model is the representation of what user thinks about the system. If the user is knowledgeable then more complete description of the system can be obtained than that of novice user.

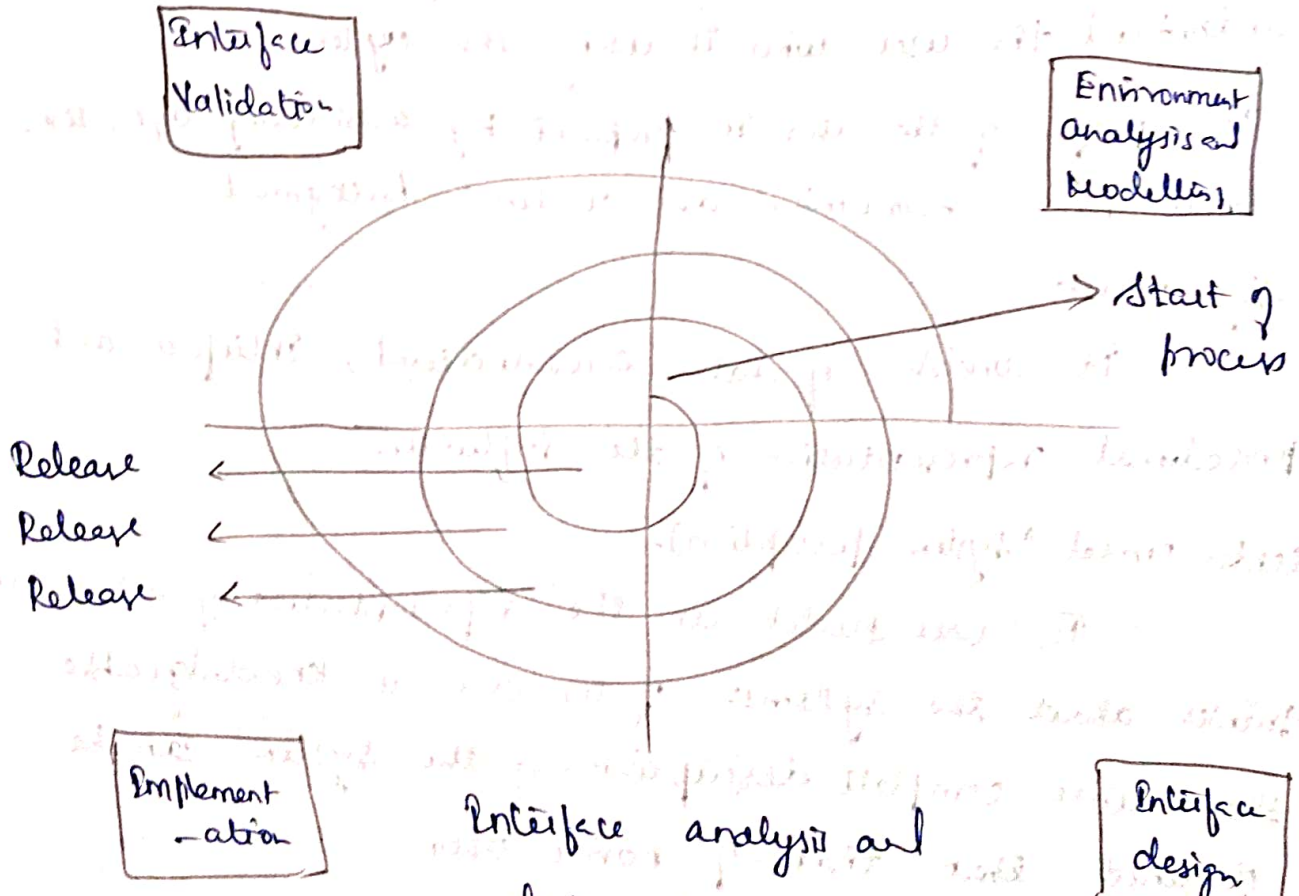
Implementation model:

The implementation model generates the look and feel of interface. This model describes the system's semantic and syntax.

The process.

The user interface analysis and design process can be implemented using iterative spiral model. It consists of four framework activities.

1. Environment analysis and modelling
2. Interface design
3. Implementation
4. Interface validation.



Interface analysis and design process.

Interface Analysis.

To perform user Interface analysis, the practioner needs to study and understand four elements.

- The users who will interact with the system through the Interface
- The tasks that end users must perform to do their work
- The content that is presented as part of the Interface
- The work environment in which these tasks will be conducted.

* The analyst strives to get the end user's mental model and the design model to converge by understanding.
* Information can be obtained from,
  * User Interviews with the end user.
  * Sales input from the sales people who interact with customer and user on a regular basis.
  * Marketing input: based on a market analysis to understand how different population segments might use the software.
  * Supporting input from the support staff who are aware of what works and what doesn't, what users like and dislike, what features generate questions & what features are easy to use.

Task Analysis and Modelling:-

Task analysis strives to know and understand

- The work the user performs in specific circumstances
- The task and subtasks that will be performed as the user does the work.
- The specific problem domain objects that the user manipulates as work is performed.
- The sequence of work tasks (i.e the workflow)
- The hierarchy of tasks.

Use cases:

- Show how an end user performs some specific work related task
- Enable the software engineer to extract tasks, objects and overall workflow of the interaction
- Helps the software engineer to identify additional helpful features

Content Analysis:

- The display content may range from character-based reports, to graphical display, to multimedia information.
- Display content may be
  - Generated by components in other parts of the application.
  - Acquired from data stored in a database that is accessible from the application

Work Environment Analysis:-

- Software products need to be designed to fit into the work environment, otherwise they may be difficult or frustrating to use

- Factors to consider include.

* Display size and height
* Keyboard size, height and ease of use
* Mouse type and ease of use
* Surrounding noise
* Space limitation for computer and/or user
* Temperature or posture restrictions.

User Interface Design:-

- user Interface design is an iterative process, where each iteration elaborates and refines the informations developed in the preceding step

General steps for user Interface design.

1) using information developed during user interface analysis, define user Interface objects and action (operations).

2) Define events (user action) that will cause the state of the user Interface to change, model this behavior.

- Depicts each Interface state as it will actually look to the end user.

Golden rules of User Interface:-

- Model how the Interface will be implemented
- Consider the computing environment (eg - display technology, Operating System) that will be used.

Design and prototype Evaluation.

- Before prototyping occurs, a number of evaluation criteria can be applied during design reviews to the design model itself.
  - The amount of learning required by the users.
  - The Interaction time and overall efficiency
  - The memory load on users.
- The complexity of the Interface and the degree to which it will be accepted by the user.
  - The prototype is evaluated for
    - * Satisfication of user requirements
    - * Conformance of to the three golden rules of User Interface design.